

Scene-Aware Conversational ADAS with Generative AI for Real-Time Driver Assistance

Kyungtae Han¹, Yitao Chen¹, Rohit Gupta¹, Onur Altintas¹

Abstract—While autonomous driving technologies continue to advance, current Advanced Driver Assistance Systems (ADAS) remain limited in their ability to interpret scene context or engage with drivers through natural language. These systems typically rely on predefined logic and lack support for dialogue-based interaction, making them inflexible in dynamic environments or when adapting to driver intent. This paper presents Scene-Aware Conversational ADAS (SC-ADAS), a modular framework that integrates Generative AI components including large language models, vision-to-text interpretation, and structured function calling to enable real-time, interpretable, and adaptive driver assistance. SC-ADAS supports multi-turn dialogue grounded in visual and sensor context, allowing natural language recommendations and driver-confirmed ADAS control. Implemented in the CARLA simulator with cloud-based Generative AI, the system executes confirmed user intents as structured ADAS commands without requiring model fine-tuning. We evaluate SC-ADAS across scene-aware, conversational, and revisited multi-turn interactions, highlighting trade-offs such as increased latency from vision-based context retrieval and token growth from accumulated dialogue history. These results demonstrate the feasibility of combining conversational reasoning, scene perception, and modular ADAS control to support the next generation of intelligent driver assistance.

I. INTRODUCTION

Advanced Driver Assistance Systems (ADAS) have become integral to modern vehicles, enhancing safety and driving comfort through features such as adaptive cruise control, lane keeping, and collision avoidance. However, current ADAS implementations are largely rule-based and reactive, offering limited flexibility in dynamic driving environments. They lack the ability to engage drivers through natural language conversation, interpret evolving road scenes, or adapt behavior in real time based on situational context and driver intent.

To enable more responsive and intuitive driver assistance, future systems must move beyond static, rule-based logic toward models that understand context and support natural interaction. As automation progresses, maintaining clear communication between the vehicle and driver is essential for safety and trust. Drivers must stay informed and engaged, confident in the system’s behavior. Intelligent interfaces that interpret natural language, reason over environmental cues, and respond conversationally can bridge the gap between human intent and machine action. Achieving this requires a unified framework combining language understanding, real-time scene perception, and structured control.

¹InfoTech Labs, Toyota Motor North America, Mountain View, CA 94043, USA. Email: {kt.han, yitao.chen, rohit.gupta, onur.altintas}@toyota.com

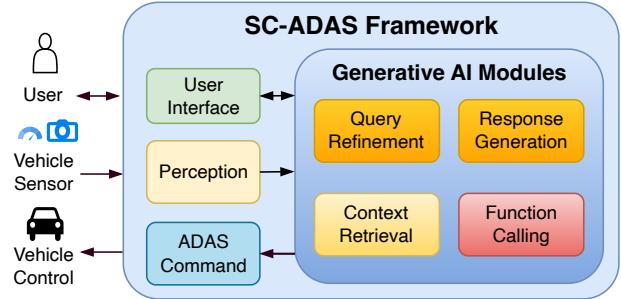


Fig. 1: Overview of SC-ADAS. The system combines four Generative AI modules with user interface, sensors, and control for real-time, scene-aware conversational interaction.

While prior works have explored components of this challenge, key limitations remain. Personalized ADAS systems [1], [2] emphasize behavioral modeling but lack interactive and scene-aware capabilities. Current in-vehicle voice assistants primarily support basic command interfaces related to navigation, media, and settings, without multi-turn dialogue or deep integration with ADAS functionalities. Tool-augmented large language models (LLMs), such as ReAct [3] and ToolLLM [4], enable structured function calling but have not yet been deployed in driving systems. Vision-language models (VLMs), including BLIP-2 [5] and GPT-4V [6], extract semantic scene information but are not used in closed-loop interactive pipelines. Recent efforts such as LaMPilot [7] and *Drive As You Speak* [8] introduce LLM-driven decision-making for driving tasks, but do not fully address the demands of scene-aware, conversational ADAS or evaluate real-time modular system behavior.

This paper presents Scene-Aware Conversational ADAS (SC-ADAS), a modular framework that integrates LLM-based reasoning, vision-to-text scene understanding, and structured function calling into a closed-loop driver assistance system. As illustrated in Fig. 1, the system consists of four Generative AI-powered modules, integrated with the user interface, vehicle sensors, and control components to support real-time, scene-aware conversational interaction. SC-ADAS supports multi-turn dialogue, allowing drivers to issue natural language queries, receive context-aware recommendations, and confirm executable ADAS commands through structured function invocation.

This work addresses the following research question: *Can Generative AI enable real-time, scene-aware, multi-turn driver assistance that is both interpretable and efficient*

enough for practical ADAS deployment? To investigate this, we develop SC-ADAS and evaluate it in a simulated driving environment [9], analyzing end-to-end latency, token usage growth, and system behavior across different modular configurations.

The main contributions of this work are:

- Introduction of SC-ADAS, a modular framework integrating LLMs, vision-to-text models, and structured ADAS control;
- Design of a multi-turn conversational pipeline grounded in real-time visual context and driver intent;
- Empirical evaluation of latency and token usage across modular configurations, highlighting design trade-offs;
- Identification of deployment strategies for GenAI-powered ADAS, including latency reduction and memory optimization.

II. RELATED WORK

A. ADAS Systems and Personalization

Modern ADAS systems provide features such as adaptive cruise control, lane keeping, and collision avoidance, but remain largely rule-based and reactive, offering limited flexibility in dynamic driving environments. Recent research has explored personalization in driver behavior modeling [1], [2], including online prediction of lane change preferences and comprehensive reviews of personalized driving datasets and modeling approaches. However, these efforts primarily target trajectory-level or behavioral adaptation, and do not support real-time, scene-aware, conversational systems.

B. Voice Assistants in Vehicles

Early systems such as Adasa [10] demonstrated the potential for conversational interfaces to access ADAS features. However, most commercial in-vehicle assistants remain limited to basic command-driven interactions, focusing on navigation, media control, and system settings without support for multi-turn dialogue, contextual understanding, or real-time integration with ADAS functionalities [11].

C. Tool-Use LLMs and Function Calling

Recent advances in LLMs have enabled reasoning-driven tool use through prompting techniques such as ReAct [3] and ToolLLM [4]. These models can interpret user queries, invoke external functions, and generate structured outputs. However, they have not been applied in real-time driving contexts and do not integrate with vehicle sensors or control systems.

D. Vision-Language Models for Scene Understanding

VLMs, including BLIP-2 [5] and GPT-4V [6], have demonstrated strong performance in extracting semantic understanding from images and multimodal scenes. Recent surveys [12], [13] have reviewed the potential of applying VLMs to autonomous driving, highlighting opportunities in perception, planning, and driver interaction. However, current VLMs are typically used for passive perception tasks rather than being integrated into closed-loop, real-time interaction

systems for ADAS operation. Their outputs remain disconnected from active driver engagement or immediate vehicle control.

E. LLM-Based Driving Agents and Instruction Following

Several recent studies have proposed leveraging LLMs to interpret natural language commands in autonomous driving scenarios. LaMPilot [7] formulates driving as a language-program generation task, enabling symbolic action code creation in simulation environments. Personalized Autonomous Driving with LLMs [14] presents field experiments where large language models are used to personalize driving behavior based on natural language instructions. The *Drive As You Speak* framework [8] conceptualizes LLMs as decision-making cores within interactive driving architectures.

While prior research has explored personalization, voice-based interaction, and language-enabled reasoning in autonomous driving, these efforts often lack real-time modular service composition and closed-loop scene-aware interaction. In contrast, SC-ADAS uniquely integrates multi-turn dialogue, scene-grounded context understanding, structured ADAS actuation, and modular service evaluation, representing a practical step toward deploying Generative AI-powered ADAS systems in real-world environments.

III. SC-ADAS FRAMEWORK

A. Design Objectives

The SC-ADAS framework bridges the gap between natural driver input and real-time ADAS behavior by integrating perception, reasoning, and actuation in a modular conversational loop. The design objectives are as follows:

- 1) **Multimodal Context Understanding:** The system processes a driver's voice command as the primary query input and supplements it with multimodal context, including real-time sensor readings and vision-derived scene descriptions. The complete driving context is defined as:

$$C = \{C_{\text{sensing}}, C_{\text{document}}\} \quad (1)$$

where C_{sensing} includes real-time sensor data (e.g., speed, camera images), and C_{document} refers to static knowledge such as maps and prior preferences.

- 2) **Context-Aware Command Generation:** Given a refined query Q_{refined} , context C , and dialogue history H , the system generates an ADAS command set S through a learned mapping:

$$S = f(Q_{\text{refined}}, C, H) \quad (2)$$

ensuring commands align with both user intent and situational constraints.

- 3) **Low-Latency Processing:** All processing stages are required to complete within real-time constraints to maintain safe and timely driver assistance.

TABLE I: Summary of SC-ADAS Modular Components

Module	Function
Query Refinement	Interprets the driver's voice command and adds metadata such as whether context or actuation is needed.
Context Retrieval	Collects relevant multimodal information, including real-time sensor data and vision-to-text scene descriptions.
Response Generation	Crafts a natural language reply based on query intent, retrieved context, and dialogue history.
Command Generation	Translates confirmed user intent into structured ADAS commands using function calling.

B. System Architecture Overview

As illustrated in Fig. 1, the SC-ADAS system is organized as a modular pipeline composed of four Generative AI-powered components. Spoken driver queries are refined, augmented with environmental context, processed through dialogue generation, and ultimately translated into structured ADAS commands. Each module in the architecture performs a distinct role in the conversational loop. Their functions are described below.

C. Modular Pipeline Components

Table I summarizes the roles of the four primary modules.

1) Query Refinement Module: This module interprets the driver's voice command, which is converted to text via speech recognition, and produces a refined query Q_{refined} and metadata M . Metadata flags indicate whether context retrieval or actuation is required. Formally:

$$(Q_{\text{refined}}, M) = f_{\text{refine}}(Q_{\text{user}}, H) \quad (3)$$

2) Context Retrieval Module: Triggered when additional context is needed, this module gathers sensing and document data:

$$C = f_{\text{retrieval}}(Q_{\text{refined}}, M) \quad (4)$$

Vision-to-text models are used to interpret camera inputs into natural language scene descriptions.

3) Response Generation Module: Given the refined query, context, prior action set, and dialogue history, this module generates a natural language reply R :

$$R = f_{\text{response}}(Q_{\text{refined}}, C, S, H) \quad (5)$$

This stage supports recommendation, clarification, and confirmation interactions.

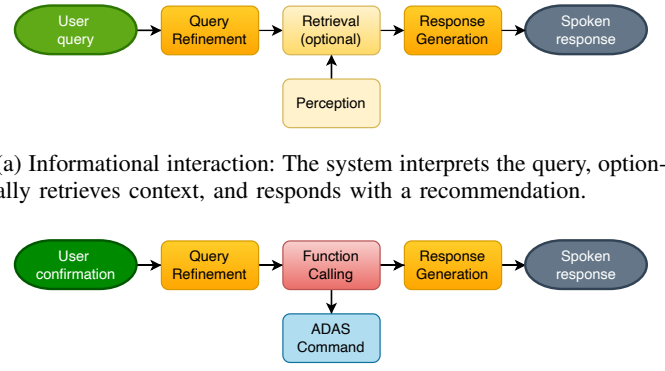
4) Command Generation Module: Upon driver confirmation, this module formats intent into structured ADAS commands, compatible with vehicle or simulator control APIs. For example:

```

1 { "name": "set_speed",
2   "arguments": { "speed": 50 } }
```

Listing 1: Example function call output.

The structured output is forwarded to the ADAS backend for execution.



(a) Informational interaction: The system interprets the query, optionally retrieves context, and responds with a recommendation.

(b) Actionable interaction: The system processes user confirmation, generates a structured command, and executes it with acknowledgment.

Fig. 2: Interaction flows supported by SC-ADAS. (a) handles general recommendations and status queries; (b) handles confirmed actions resulting in ADAS control changes.

IV. CONVERSATIONAL FLOWS

A. Overview of Interaction Patterns

SC-ADAS supports two primary types of conversational interaction: *Informational* and *Actionable*. Both flows leverage the modular reasoning and response pipeline described earlier, adapting processing paths based on user intent.

As illustrated in Fig. 2:

(a) Informational Interaction: The driver issues a query requesting a recommendation or status update. The system refines the query, optionally retrieves real-time context (e.g., current speed, scene description), and generates a natural language response.

(b) Actionable Interaction: The driver confirms an action that alters ADAS behavior. The system refines the confirmed intent, generates a structured command via function calling, and executes the command after validation.

This modular routing enables SC-ADAS to dynamically adjust its reasoning and control steps while ensuring that all vehicle-level actuation is explicitly confirmed before execution.

B. Example of Multi-Turn Conversational Flow

To illustrate SC-ADAS operation during runtime, we present a two-turn scenario involving both interaction types. Each turn selectively activates different modules based on the query's metadata and system context.

Turn 1: Informational Request:

- Driver Query:** "Could you recommend a safe speed for this road?"
- Query Refinement:** The system refines the input to: $Q_{\text{refined}} = \text{"What speed is safe for this road?"}$ with metadata $M = \{\text{sensing_required} = \text{true}\}$.
- Context Retrieval:** Retrieved context includes: $C_{\text{vision}} = \text{"urban road, moderate traffic"}$ and $C_{\text{motion}} = \text{"current speed 42 MPH"}$.
- Response Generation:** Based on the retrieved context and dialogue history, the system responds: "Based on

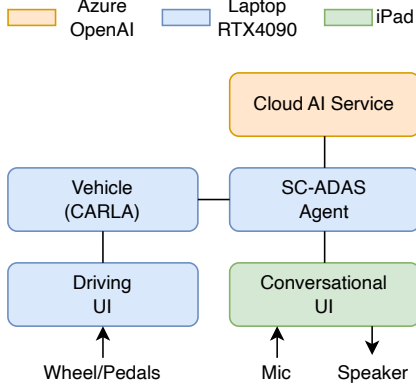


Fig. 3: SC-ADAS system implementation architecture. The laptop runs the CARLA simulator, SC-ADAS agent, and UI server. The iPad serves as the conversational interface. GPT-4o is accessed remotely via Azure OpenAI using prompt-based interaction.

the detected urban road and moderate traffic conditions, I recommend setting the speed to 40 MPH. Would you like to apply this setting?”

Turn 2: Command Confirmation (Actionable Interaction):

- 5) **Driver Query:** “Yes, go ahead.”
- 6) **Query Refinement:** Refined as: $Q_{\text{refined}} =$ “Yes, set the speed to 40 MPH” with metadata $M = \{\text{actuation_required} = \text{true}\}$.
- 7) **Command Generation:** Following the structure shown in Listing 1, the system generates a `set_speed` command with the argument `speed = 40`.
- 8) **Command Execution and Acknowledgment:** The command is transmitted to the ADAS backend for execution, and the system acknowledges with: “Speed has been set to 40 MPH.”

V. SYSTEM DEPLOYMENT

A. System-Level Deployment

The system-level implementation architecture of SC-ADAS is illustrated in Fig. 3. The framework operates entirely on a high-performance laptop equipped with an NVIDIA RTX 4090 GPU and Intel Core i9 CPU, which hosts the CARLA simulator, vehicle control modules, and the SC-ADAS agent.

The driver interacts with SC-ADAS through an iPad running a browser-based conversational interface. The iPad captures voice queries via its built-in microphone, renders system responses using on-device text-to-speech, and displays the query-response history. Communication between the iPad and the backend agent is conducted over a local Wi-Fi network using WebSocket and HTTP protocols.

The SC-ADAS agent manages all four modular components described in Section III, namely query refinement, context retrieval, response generation, and command generation.



Fig. 4: Testing setup using CARLA simulator and browser-based user interface on an iPad. The iPad handles voice input, speech output, and dialogue display.

Each module is implemented as a prompt-based reasoning service interacting with GPT-4o hosted on Azure OpenAI. Prompts are dynamically constructed based on driver queries, sensor context, and conversation history, enabling flexible orchestration of modular reasoning.

This architecture balances local perception and control with cloud-based reasoning, supporting real-time operation without requiring on-device model inference. The system achieves low-latency interaction while leveraging state-of-the-art Generative AI capabilities for scene-aware dialogue and adaptive ADAS command generation.

B. Simulation Environment

SC-ADAS is implemented and tested using the CARLA Simulator (v0.9.15) [9], providing high-fidelity simulated sensor data, road scene diversity, and scripted driving behaviors. The ego vehicle is equipped with simulated telemetry, including real-time speed, and front-facing camera images, all streamed to the SC-ADAS backend for reasoning and actuation.

C. User Interface Setup

The complete experimental setup, including the iPad conversational interface and simulator, is shown in Fig. 4. The iPad interface supports the following functionalities:

- **Speech Recognition:** Converts spoken driver queries into text using the browser-based Web Speech API.
- **Text-to-Speech:** Delivers SC-ADAS system responses as spoken messages using the built-in Web Speech Synthesis API.
- **Dialogue Display:** Presents real-time system responses, context information, and status updates on the screen.

Communication between the iPad and the backend is maintained over a low-latency Wi-Fi connection using WebSocket for streaming telemetry updates and HTTP for cloud-based API transactions.

D. Backend Processing

The backend server orchestrates local data preparation and cloud-based reasoning as follows:

- **Local Preprocessing Services:** Local modules collect real-time telemetry data (e.g., ego speed, vehicle state, front-facing camera images) and process spoken driver input into text format. This raw information is organized into structured prompt templates, including the driver's query, live sensing data, and minimal dialogue history.
- **Cloud-Based Reasoning and Generation:** All reasoning and decision-making modules, including query refinement, metadata extraction (e.g., sensing or actuation requirements), context retrieval, response generation, and command generation, are executed through a cloud-hosted Generative AI model. Specifically, GPT-4o, accessed via Azure OpenAI endpoints, performs natural language reasoning, environmental grounding, and structured function calling.

At each interaction turn, the backend server constructs a prompt that encapsulates the refined conversation history and relevant sensor data, then submits this prompt to the cloud model for processing. The model interprets the query, retrieves and fuses context if necessary, generates conversational responses, and constructs structured ADAS commands when confirmed.

To improve responsiveness, the maximum output token length is limited to 300 tokens for standard interactions and extended to 500 tokens when vision-derived contextual information is incorporated.

E. System Integration and Communication

The overall communication architecture is designed to support modular, real-time interaction between components:

- ZeroMQ is used for telemetry data streaming from the CARLA server to the backend service.
- REST API calls manage cloud interactions for Generative AI services.
- WebSocket communication handles real-time synchronization between the backend and the iPad UI.

The modular communication strategy allows each system component (simulation, backend, reasoning model, and interface) to be deployed flexibly across different machines or scaled independently if needed.

VI. EVALUATION

A. Interaction Types Observed

To evaluate SC-ADAS across different real-world usage scenarios, we ran a continuous interactive driving session and later categorized the observed interactions into three primary types:

- **Conversational-Only Service:** Driver queries are processed into natural language responses without ADAS actuation or scene context retrieval.
- **Conversational ADAS Service:** Structured ADAS commands are issued upon driver confirmation, without using vision-derived context.

- **Scene-Aware Conversational Service:** The system processes driver queries by retrieving real-time vision context, interpreting the scene, and generating a context-aware conversational response, without issuing ADAS control commands.

All interaction types share the Refiner and Responder modules, enabling isolated analysis of the Vision and Actuator module impacts.

B. Evaluation Scenarios

To comprehensively evaluate system performance, we conducted multiple interaction turns in a simulated driving session. These included scene-related queries (e.g., identifying objects in the environment), speed recommendation requests for adaptive cruise control (ACC), and confirmation commands for setting ADAS parameters. This mix of informational and actionable interactions ensures that all major system capabilities are exercised under realistic conditions.

SC-ADAS is evaluated along two dimensions: scene understanding effectiveness and real-time modular service efficiency.

1) *Scene-Based Speed Recommendation Evaluation:* To assess scene-aware reasoning, we design a 2×3 matrix combining two road types (highway in Town4 and downtown in Town3) and three weather conditions (clear, rainy, and foggy). In each case, the system receives the spoken query: "What speed do you recommend for this road?"

Front-camera images captured from the CARLA Simulator were used to simulate these conditions. A screenshot from each of the six test cases is shown in Fig. 5.

2) *Scene Consistency Evaluation:* To assess the consistency of SC-ADAS recommendations in repeated contexts, we designed an additional evaluation in which the ego vehicle returned to visually similar scenes during the same driving session. At each revisit, the driver issued the same adaptive cruise control query to observe whether the system would produce consistent responses.

This scenario helps test the stability of vision-based interpretation and downstream reasoning in a multi-turn dialogue setting, where prior user confirmations or preferences might reasonably influence future responses. Differences in output can reveal how variability in scene descriptions affects recommendation generation.

3) *Modular Service Performance Evaluation:* For real-time performance evaluation, we conduct full conversational service invocations across the two service types, including both informational (recommendations) and actionable (ADAS setting) interactions. This enables measurement of system latency and token usage evolution across different service complexities.

C. Evaluation Metrics

We define two primary evaluation metrics:

- **Module Processing Latency:** For each service invocation during simulated driving sessions, we measured the processing time of each reasoning module (Refiner, Vision, Actuator, Responder) as the duration between

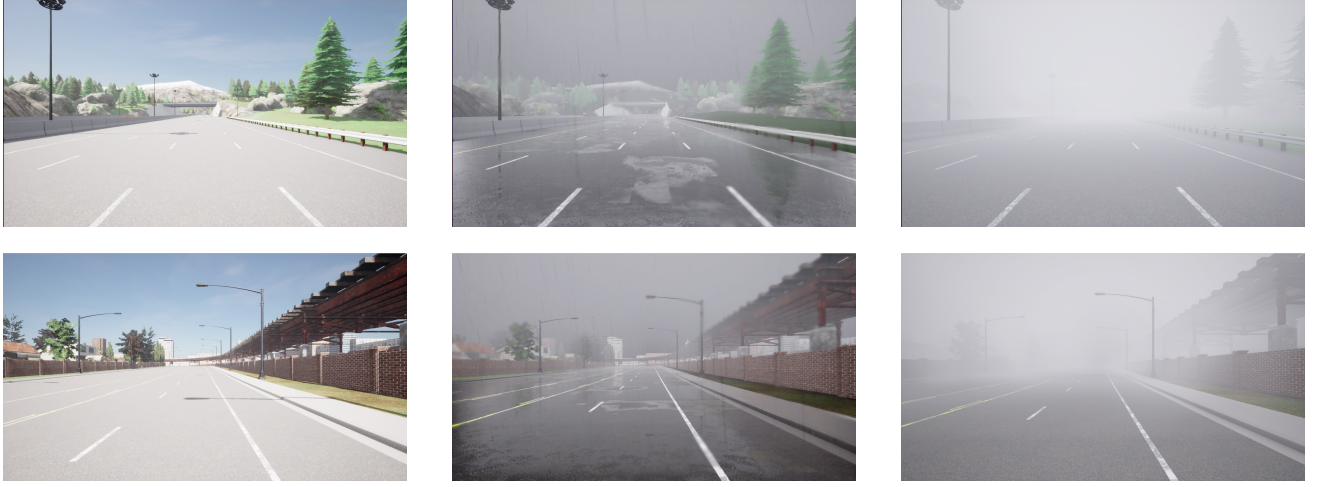


Fig. 5: Scene-based evaluation scenarios in CARLA Simulator: clear, rainy, and foggy conditions on highway (top row) and downtown (bottom row) roads. Each screenshot corresponds to a test instance where SC-ADAS provides speed recommendations.

sending a prompt to the cloud-based Generative AI and receiving a response. Front-end processing times, such as speech recognition and UI rendering, were excluded as their contribution was negligible.

- **Token Usage:** Measured as input (uplink) and output (downlink) token counts per service invocation, tracked separately for each reasoning module.

These metrics capture system responsiveness, cloud service demands, and scalability characteristics.

VII. RESULTS AND DISCUSSION

We evaluate SC-ADAS from two perspectives: its effectiveness in producing scene-aware driver assistance, and its efficiency in real-time operation under different modular service configurations.

A. Effectiveness: Scene Understanding and Decision Quality

1) *Scene-Based Speed Recommendation:* SC-ADAS produced differentiated speed recommendations across the six test scenarios shown in Fig. 5. Table II summarizes the recommended speeds for each combination of road type and weather condition.

TABLE II: Recommended Speed (MPH) Across Road and Weather Conditions

Road Type	Clear	Rainy	Foggy
Highway (CARLA Town4)	65	40	30
Downtown (CARLA Town3)	45	20	20

The results demonstrate that SC-ADAS’s vision-to-text module successfully interprets environmental conditions, enabling adaptive and context-aware speed recommendations. The outputs reflect human-like sensitivity to road type and visibility, validating the system’s ability to ground language understanding in real-world scenes.

2) *Consistency Across Multi-Turn Scene Queries:* To evaluate the consistency of recommendations over time, we conducted additional tests where the driver returned to similar scenes within the same driving session and issued follow-up queries. While the system correctly interpreted the environment and produced relevant responses, we observed slight variations in the recommended ACC settings across turns, even when the vehicle revisited similar locations under comparable visual conditions.

This variation is likely due to the generative nature of the vision-to-text module, where subtle differences in textual scene descriptions influenced downstream reasoning. Although each description was semantically valid, wording variability introduced minor fluctuations in the resulting recommendations.

This observation highlights a trade-off between natural language flexibility and behavioral consistency. To improve stability, future iterations of SC-ADAS may incorporate more structured and persistent scene representations (e.g., scene embeddings, map-based anchors, or traffic state descriptors). These enhancements could enable the retrieval of prior user confirmations or preferences in similar contexts, fostering more consistent and personalized ADAS support.

B. Efficiency: Latency and Resource Usage

1) *Latency Analysis Across Service Types:* We evaluated the end-to-end reasoning latency across three modular service types:

- **Conversational-Only Service:** Refiner + Responder
- **Conversational ADAS Service:** Refiner + Actuator + Responder
- **Scene-Aware Conversational Service:** Refiner + Vision + Responder

For clarity, we refer to the Context Retrieval module (vision-to-text processing) as the Vision module, and the Command Generation module (structured ADAS function calling) as the Actuator module in the following analysis.

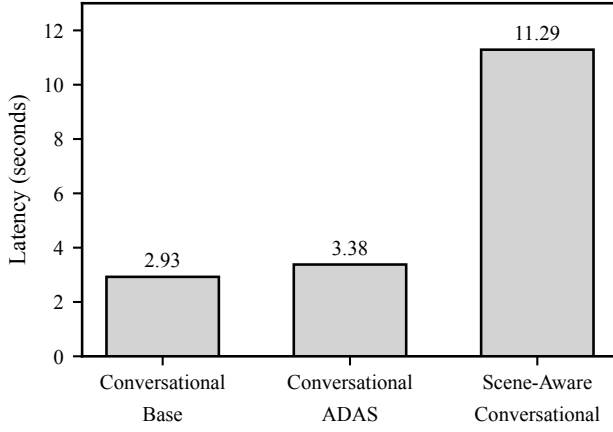


Fig. 6: Latency comparison across different service configurations. Scene-aware vision processing introduces significant additional latency, while command generation overhead remains minimal.

All service types share the Refiner and Responder modules, providing a consistent baseline for comparing the impact of the Vision and Actuator modules with minimal confounding factors.

As shown in Fig. 6, the Scene-Aware Service exhibited an average latency of approximately 11 seconds, introducing an additional 8 seconds compared to the base Conversational-Only Service (3 seconds). The Conversational ADAS Service maintained similar latency to the baseline, indicating that structured function generation incurs minimal additional delay. The increased latency is primarily attributable to the Vision module’s image processing and scene interpretation workload.

2) *Latency Breakdown per Service:* Fig. 7 presents per-service latency decomposition across agent modules. Interaction sequences correspond to sequential conversational turns simulated during driving. The Refiner and Responder modules consistently contribute 2–5 seconds. The Actuator module adds approximately 1 second. Vision processing introduces the most variability, with end-to-end latencies ranging from 8 to 17 seconds, particularly in visually complex scenes.

3) *Token Usage Growth and Context History:* Token usage analysis (Fig. 8) shows that total token volume, particularly for input (uplink) tokens, grows over time as dialogue history accumulates. Output (downlink) tokens remain relatively stable across services, typically under the 500-token maximum configuration.

4) *Token Distribution Across Agents:* To better understand resource demands per module, we further analyzed input and output token distributions. Fig. 9 shows the breakdown across 23 complete service invocations.

The Refiner module consistently accounted for the majority of input tokens due to conversation history management. The Vision module contributed additional input and output tokens when scene interpretation was triggered.

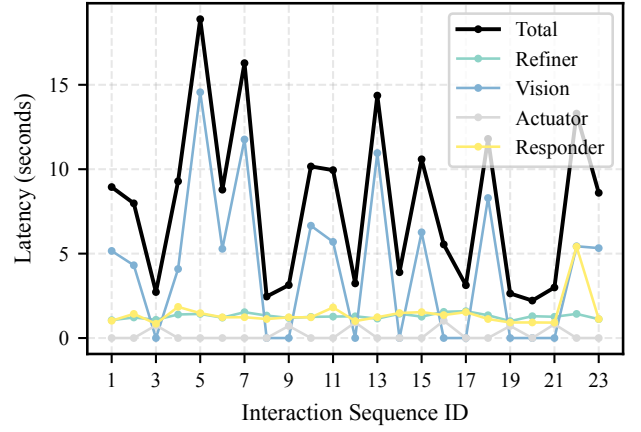


Fig. 7: Latency decomposition across interaction sequences, showing the contribution of each module (Refiner, Vision, Actuator, Responder) to the total service processing time.

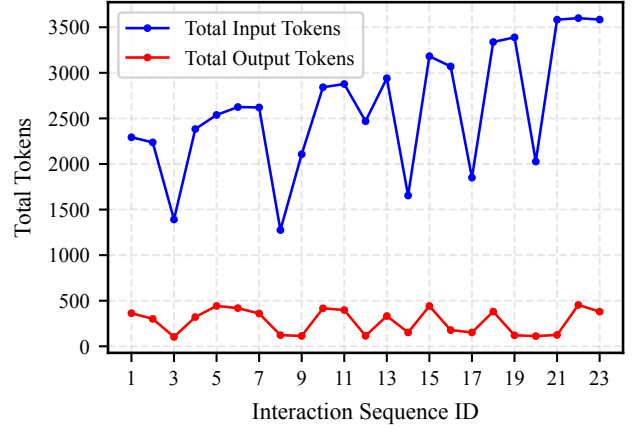
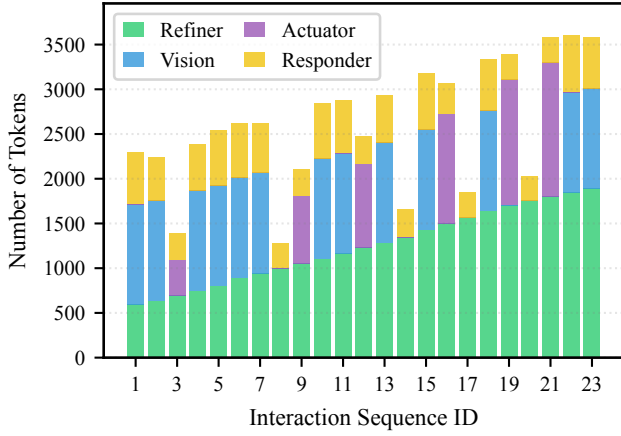


Fig. 8: Analysis of total input (uplink) and output (downlink) token usage across interaction sequences, showing input token growth driven by conversation history.

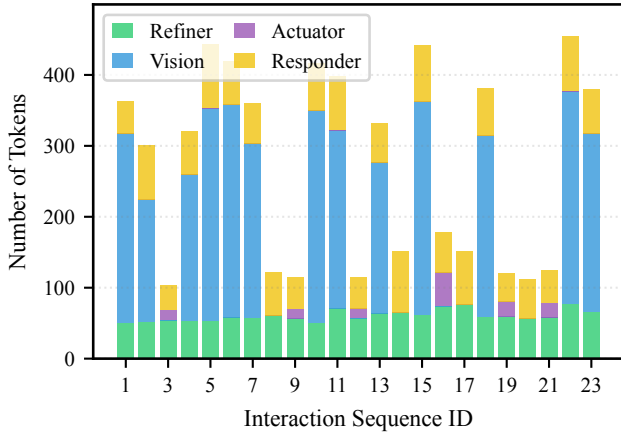
Output token volume remained significantly lower than input volume, with the Vision module contributing the highest peaks during image-grounded interactions. The Refiner and Responder modules consistently generated moderate output tokens across most interactions.

VIII. FUTURE WORK

Several directions will extend the capabilities of SC-ADAS. First, we plan to deploy lightweight vision and language models on embedded platforms to reduce cloud latency and minimize uplink token transmission. Second, to improve contextual consistency and preference retention, we will integrate Retrieval-Augmented Generation to retrieve past driver interactions and system responses in visually or semantically similar scenes. Finally, we will explore output validation techniques to ensure the safety and reliability of GenAI-generated decisions in real-world applications.



(a) Input tokens per per interaction sequence. The Refiner dominates due to conversation context accumulation.



(b) Output tokens per per interaction sequence. The output token volume relatively remained low. The vision module contributed additional output tokens.

Fig. 9: Input and output token distributions across agent modules over 23 service invocations.

IX. CONCLUSION

This paper presented SC-ADAS, a modular framework that integrates Generative AI components, including LLM reasoning, vision-to-text interpretation, and structured function calling to enable real-time, scene-aware driver assistance. The system supports multi-turn dialogue, enabling context-grounded recommendations and ADAS control through natural language. Evaluation in a simulated environment showed that SC-ADAS effectively interprets diverse road and weather conditions to generate adaptive speed recommendations. The results also reveal trade-offs introduced by scene-aware processing, including increased latency from vision-based context retrieval and uplink token growth due to accumulated dialogue history. Additionally, slight variations in recommendations across revisited scenes suggest the benefit of more structured scene representations to improve consistency. Overall, SC-ADAS demonstrates the feasibility of combining conversational reasoning, visual context understanding, and

modular ADAS control for interpretable and adaptive driver support.

ACKNOWLEDGMENT

The contents of this work only reflect the views of the authors, who are responsible for the facts and the accuracy of the data presented herein. The contents do not necessarily reflect the official views of Toyota Motor North America.

REFERENCES

- [1] X. Liao, X. Zhao, Z. Wang, Z. Zhao, K. Han, R. Gupta, M. J. Barth, and G. Wu, "Driver digital twin for online prediction of personalized lane change behavior," *IEEE Internet of Things Journal*, vol. 10, no. 18, pp. 16 253–16 266, 2023.
- [2] X. Liao, Z. Zhao, M. J. Barth, A. Abdelraouf, R. Gupta, K. Han, J. Ma, and G. Wu, "A review of personalization in driving behavior: Dataset, modeling, and validation," *IEEE Transactions on Intelligent Vehicles*, pp. 1–22, 2024.
- [3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.
- [4] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, dahai li, Z. Liu, and M. Sun, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," in *The Twelfth International Conference on Learning Representations*, 2024.
- [5] J. Li, D. Li, S. Savarese, and S. Hoi, "BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models," in *International conference on machine learning*. PMLR, 2023, pp. 19 730–19 742.
- [6] OpenAI, "GPT-4 technical report," <https://openai.com/research/gpt-4>, 2023.
- [7] Y. Ma, C. Cui, X. Cao, W. Ye, P. Liu, J. Lu, A. Abdelraouf, R. Gupta, K. Han, A. Bera, J. M. Rehag, and Z. Wang, "LaMPilot: An open benchmark dataset for autonomous driving with language model programs," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [8] C. Cui, Y. Ma, X. Cao, W. Ye, and Z. Wang, "Drive as you speak: Enabling human-like interaction with large language models in autonomous vehicles," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2024, pp. 902–909.
- [9] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Conference on Robot Learning*, 2017, pp. 1–16.
- [10] S.-C. Lin, C.-H. Hsu, W. Talamonti, Y. Zhang, S. Oney, J. Mars, and L. Tang, "Adasa: A conversational in-vehicle digital assistant for advanced driver assistance features," in *Proceedings of the 31st Annual ACM Symposium on User Interface Software and Technology*, 2018, pp. 531–542.
- [11] H. Du, X. Feng, J. Ma, M. Wang, S. Tao, Y. Zhong, Y.-F. Li, and H. Wang, "Towards proactive interactions for in-vehicle conversational assistants utilizing large language models," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, ser. IJCAI '24, 2024.
- [12] X. Zhou, M. Liu, E. Yurtsever, B. L. Zagar, W. Zimmer, H. Cao, and A. C. Knoll, "Vision language models in autonomous driving: A survey and outlook," *IEEE Transactions on Intelligent Vehicles*, pp. 1–20, 2024.
- [13] C. Cui, Y. Ma, X. Cao, W. Ye, Y. Zhou, K. Liang, J. Chen, J. Lu, Z. Yang, K.-D. Liao, T. Gao, E. Li, K. Tang, Z. Cao, T. Zhou, A. Liu, X. Yan, S. Mei, J. Cao, Z. Wang, and C. Zheng, "A survey on multimodal large language models for autonomous driving," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV) Workshops*, January 2024, pp. 958–979.
- [14] C. Cui, Z. Yang, Y. Zhou, Y. Ma, J. Lu, L. Li, Y. Chen, J. Panchal, and Z. Wang, "Personalized autonomous driving with large language models: Field experiments," in *2024 IEEE 27th International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 2024, pp. 20–27.